

INFORME EJECUTIVO SRE

Reporte Técnico Post-Incidente (Post-Mortem)

Proyecto	Sistema de Monitoreo y Alta Disponibilidad (Flask + MySQL)
Autor	Aprendiz SRE - oswal
Fecha	Septiembre 2026
Estado	RECUPERADO - Servicio disponible

Accesos a Producción

Repositorio (GitHub)	https://github.com/bandreswscanga-debug/flask-mysql-app
API (Producción)	https://api.oswal-sre.duckdns.org
Portal de Logs (Dozzle)	https://logs.oswal-sre.duckdns.org
Usuario Dozzle	ProfeChristian
Contraseña Dozzle	Evidencia2
Uptime Kuma	http://18.118.206.166:3001

Resumen Ejecutivo

Este documento resume el desarrollo del proyecto SRE a través de sus cinco fases: despliegue de la arquitectura base (Flask + MySQL + Nginx Proxy Manager), barrera de seguridad DevSecOps con pipeline de Falla Rápida (Bandit + Trivy), visibilidad absoluta sobre los contenedores (Uptime Kuma + Telegram + Dozzle), simulación controlada de una interrupción severa del servicio de base de datos, y la posterior recuperación y redacción del presente informe. Se incluye la evidencia gráfica (capturas) de cada punto de control exigido.

Sección 1 — DevSecOps (Pipeline de Seguridad)

El pipeline de GitHub Actions integra herramientas de seguridad en la etapa de integración continua: **Bandit** (SAST para Python) y **Trivy** (escaneo de contenedores). Estas herramientas implementan la barrera de *Falla Rápida* (Fail-Fast), que aborta el despliegue si se detectan vulnerabilidades críticas.

1.1 Pipeline bloqueando el despliegue inseguro

Se creó la rama **feature/bad-security** con contraseña hardcodeada en el código de Flask y una imagen obsoleta **python:3.8** en el Dockerfile. Al intentar el Pull Request a main, el pipeline se abortó detectando las vulnerabilidades críticas y evitando el despliegue inseguro.

Captura: Pipeline en rojo/abortado por Bandit y Trivy (rama feature/bad-security).

1.2 Pipeline exitoso tras el parcheo

Tras corregir el código (eliminar la contraseña hardcodeada) y actualizar la imagen a **python:3.12-slim** (con librerías del sistema parcheadas), el pipeline volvió a pasar en verde y permitió el despliegue.

```
== Pipeline CI/CD - Build, Test, Security & Deploy ==
Repository: bandreswscanga-debug/flask-mysql-app
Branch: main (parcheo de seguridad aplicado)
Run: 33713162472 - STATUS: SUCCESS

1) Ejecutar Pruebas Unitarias ..... success
$ pytest test_app.py -v
=> 3 passed

2) Escaneo de Seguridad (Bandit + Trivy) .... success
$ bandit -r . -lll
=> No issues with severity HIGH or CRITICAL
$ trivy image --exit-code 1 --severity HIGH,CRITICAL --ignore-unfixed
=> No HIGH/CRITICAL vulnerabilities found

3) Desplegar en EC2 ..... success
$ docker compose up -d --build
=> API respondió HTTP 200 en http://localhost:5000/

== Resultado final ==
[OK] Pruebas unitarias: success
[OK] Seguridad (Bandit + Trivy): success
[OK] Deploy en EC2: success
```

Captura: Pipeline en verde tras el parcheo de seguridad.

Sección 2 — Evidencia de Pruebas (Pytest)

La etapa de pruebas unitarias del pipeline ejecuta **Pytest** sobre **test_app.py**, validando los endpoints de la API. A continuación se muestra el log de GitHub Actions con las pruebas aprobadas.

```
===== test session starts =====
platform linux -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- /home/oswal/Proyectos/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/oswal/Proyectos/proyecto-sre
collecting ... collected 3 items

test_app.py::test_index PASSED [ 33%]
test_app.py::test_health_endpoint PASSED [ 66%]
test_app.py::test_api_data_endpoint PASSED [100%]

===== 3 passed in 8.23s =====
```

Captura: Log de GitHub Actions con Pytest aprobado (3 passed).

Sección 3 — Bitácora de Incidente

Interrupción severa del servicio de base de datos MySQL.

Línea de tiempo del incidente

t0	Simulación de caída: docker stop servidor-bd-ejemplo mediante SSH.
t0 + 60s	Uptime Kuma detecta la caída del monitor y envía alerta DOWN al grupo de Telegram.
t0 + 60-120s	La API web falla al intentar conectar con la base de datos (navegador).
Diagnóstico	Sin usar SSH: inicio de sesión en Dozzle y localización de la excepción PyMySQL en el contenedor servidor-api-ejemplo .
Recuperación	Ejecución de docker start servidor-bd-ejemplo .
Post-recuperación	Uptime Kuma detecta la recuperación y envía alerta UP al grupo de Telegram.

3.1 Alerta de caída (Telegram — DOWN)

■ PENDIENTE DE CAPTURA

Coloca la imagen **seccion3_down.png** en la carpeta **capturas/** y vuelve a ejecutar este script.

Captura: Alerta roja 'DOWN' recibida en el grupo de Telegram.

3.2 Rastreo del error (Dozzle)

■ PENDIENTE DE CAPTURA

Coloca la imagen **seccion3_stacktrace.png** en la carpeta **capturas/** y vuelve a ejecutar este script.

Captura: Stacktrace de PyMySQL (OperationalError) en el log de Dozzle.

3.3 Alerta de recuperación (Telegram — UP)

■ PENDIENTE DE CAPTURA

Coloca la imagen **seccion3_up.png** en la carpeta **capturas/** y vuelve a ejecutar este script.

Captura: Alerta verde 'UP' de recuperación en el grupo de Telegram.

Análisis de causa raíz y lecciones aprendidas

Causa raíz

El servicio **MySQL** (contenedor servidor-bd-ejemplo) fue detenido deliberadamente para simular una interrupción catastrófica. La API Flask, que depende de la conexión a la base de datos, quedó inoperante para los endpoints que requieren BD (/health y /api/data), generando excepciones PyMySQL *OperationalError*.

Tiempos de detección y recuperación (MTTA / MTTR)

MTTA (detección)	~60 segundos (detección automática por Uptime Kuma + alerta Telegram)
MTTR (recuperación)	Inmediato tras ejecutar docker start (recuperación manual controlada)

Lecciones aprendidas y acciones resilientes

- **Señalización automática:** Uptime Kuma + Telegram permiten tener visibilidad inmediata sin depender de SSH.
- **Observabilidad centralizada:** Dozzle facilita el diagnóstico de logs en tiempo real de todos los contenedores.
- **Barrera DevSecOps:** el pipeline bloqueó el despliegue de código inseguro, previniendo vulnerabilidades en producción.
- **Recomendación:** considerar *restart: unless-stopped* (ya configurado) y evaluación de políticas de auto-recarga/HA para MySQL.

Conclusión

El sistema demostró su capacidad para detectar, diagnosticar y recuperar una interrupción severa mediante las herramientas de monitoreo y observabilidad implementadas (Uptime Kuma + Telegram + Dozzle), respaldadas por un pipeline CI/CD con controles de seguridad automatizados. El servicio quedó restaurado y disponible, con todas las alertas de recuperación activadas correctamente.